

VisionGuard: Intelligent CCTV-Based Employee Identification and Suspicious Activity Monitoring System

Project Proposal

V 1.0

By

1	Aqsa Aziz	NUM-BSCS-2023-22
2	Iqra Gul	NUM-BSCS-2023-35
3	Aliya Ashraf	NUM-BSCS-2024-08
4	Areeba Tahir	NUM-BSCS-2024-15



Department of Computer Sciences

Namal University

Mianwali, Pakistan

Submission Date : 9th October, 2026

REVISION HISTORY

Date	Version	Description	Approved by
9 Oct 2026	V 1.0	Initial version of the project proposal.	

TABLE OF CONTENTS

1. INTRODUCTION:	5
2. PROBLEM STATEMENT:.....	5
3. PROJECT OBJECTIVES:	6
4. PROJECT SCOPE:	7
5. STAKEHOLDER IDENTIFICATION:	8
6. SOFTWARE DEVELOPMENT METHODOLOGY:	8
7. TOOLS AND TECHNOLOGIES:.....	12
REFERENCES	13

REQUIREMENT PROVIDER AGREEMENT

This agreement is made between the Requirement Provider and the Project Team for the development of the proposed software project titled:

Project Title: **VisionGuard: Intelligent CCTV-Based Employee Identification and Suspicious Activity Monitoring System**

The Requirement Provider will provide the necessary project information, domain knowledge, user needs, and other relevant details required for understanding the proposed system and will review and validate the information documented by the Project Team. The Project Team will accurately document the agreed project scope and develop the system based on the requirements established and approved during the subsequent requirements engineering process. Any significant changes to the agreed project scope will be discussed with the Requirement Provider and Project Team before implementation. Both parties agree to maintain effective communication and cooperation throughout the project.

Requirement Provider

Name: Mr. Ammar Ahmad Khan

Designation: Lecturer

Organization: Namal University Mianwali

Email: ammar.ahmad@namal.edu.pk

Signature with Date:

Project Team Representative

Name: Aqsa Aziz

Roll No: NUM-BSCS-2023-22

Email: bscs23f22@namal.edu.pk

Signature with Date: Aqs@9/10/2026

1. INTRODUCTION:

Physical security is an important concern for organizations such as industrial companies, offices, and campuses, where people, equipment, and restricted areas must be protected around the clock. Most of these places already have CCTV cameras at entry gates, corridors, work areas, and storage areas. However, the footage is mostly checked by people after something has happened, and the recordings do not directly show who was involved. After an incident, security staff must watch hours of video and collect details from several recordings, which takes a lot of time and effort.

Recent progress in computer vision and deep learning has made it possible to recognize people in surveillance video automatically [1] and to detect unusual events without watching every recording [2]. This project applies these techniques to industrial security and surveillance and uses software engineering practices to turn them into a reliable and usable system.

This project proposes **VisionGuard**, a web-based CCTV monitoring system that uses a few of an organization's existing CCTV cameras. VisionGuard will recognize enrolled employees from their faces, detect suspicious activities using simple rules based on location, working hours, and time spent in an area, and save every record against the employee's Employee ID and full name (or the label "Unknown" when the person is not recognized). Authorized security staff will use a web dashboard to search records, review alerts with picture evidence, and create reports. The system only raises alerts for people to review; it does not take any action against employees.

The project is developed as a Software Engineering course project within one semester (October 2026 – January 2027) and follows a complete software development life cycle, from requirements analysis to testing and delivery, using an Agile approach. The initial requirements will be discussed with the Requirement Provider, Mr. Ammar Ahmad Khan (project supervisor), and will be reviewed and validated with him again in every sprint during development. To keep the project realistic, the following assumptions are made:

Employee data: Employee details and approved face photos will be provided by volunteers who give their consent, with the approval of the supervisor and following data protection rules.

Cameras: A few existing cameras (for example, the main entrance, reception, corridors, and restricted areas) will be used. An employee can recognize only where the camera shows the face clearly.

Suspicious activities: The rules will be agreed with the Requirement Provider and, where available, with security staff.

Human review: The system only gives alerts, and security staff make all decisions.

2. PROBLEM STATEMENT:

In many organizations, CCTV cameras are used in a passive way. Security staff usually watch recorded footage by hand after an incident, which is slow and depends on the time and attention available. Because the recordings are not linked to people, the footage cannot easily be searched by people. To find out who was involved in an incident, or to collect everything about one employee, staff must go through different recordings and gather the details manually.

In addition, nothing warns the security team when a suspicious situation happens, such as an employee entering a zone without permission, being present after working hours, staying too long in one area, or an unknown person appearing in a restricted area. These situations are noticed late or not at all. The people affected are mainly the security staff and supervisors, who spend a lot of effort on manual checking, and the organization, which receives late and incomplete information about security events.

Solving this problem is important because a system that identifies employees automatically and links every suspicious event to an Employee ID and full name can make investigations faster, reduce manual work, and give security staff clear and searchable evidence, while keeping privacy protection and human review in place.

3. PROJECT OBJECTIVES:

The main aim of the project is to develop a web-based CCTV monitoring system that recognizes enrolled employees, detects suspicious activities, and saves the information against Employee ID and full name for review by authorized security staff. The specific objectives are:

- **Requirements:** Gather the initial requirements with the Requirement Provider (meetings and a review of the camera environment) and obtain an approved baseline Software Requirements Specification (SRS) and a prioritized product backlog by 25 October 2026. Later requests that are small, feasible, and do not affect the core of the project will be added to the backlog and implemented in the following sprint.
- **Employee identification:** Process video from selected CCTV cameras at about 10 frames per second per stream and identify enrolled employees by their faces, with a Rank-1 accuracy (the top match is the correct employee) of 90% or higher and a false accept rate of 1% or lower on the test footage by 3 January 2027.
- **Suspicious activity detection:** Detect four rule-based activities (entering a zone without permission, after-hours presence, loitering, and an unknown person in a restricted zone) with a recall of 85% or higher and a precision of 80% or higher on staged events by 3 January 2027.
- **Data storage:** Store every sighting and suspicious event in a central PostgreSQL database, linked to the Employee ID and full name or labeled “Unknown”, so that no record is left without an identity label.
- **Alerts and dashboard:** Show alerts on the web dashboard within 5 seconds of the event, and provide search by Employee ID or name, alert review (Pending, Confirmed, or False Alarm), and exportable reports by 3 January 2027.
- **Security and privacy:** Protect employee data with login, role-based access, audit logs, secure data transfer, and limits on how long data is kept [3].
- **Delivery:** Test the system on recorded CCTV clips, complete user acceptance testing with the Requirement Provider by 12 January 2027, and submit the working prototype, documentation, and final report by 15 January 2027.

4. PROJECT SCOPE:

The scope below is the baseline plan for one semester (October 2026 – January 2027). In line with the Agile approach, small and feasible changes requested by the Requirement Provider that do not affect the core of the project can be accepted and implemented in later sprints, while significant changes to the agreed scope will be discussed with the Requirement Provider and the project team before any implementation (see Section 6.4).

In Scope

- Recognition of enrolled employees from selected CCTV cameras, using only faces that are large, sharp, and straight enough for identification [4].
- Employee enrolment: saving the Employee ID, full name, and approved face data of each employee.
- Detection of four suspicious activities using rules based on location, working hours, and time spent in an area: entering a zone without permission, after-hours presence, staying too long in an area (loitering) [5], and an unknown person in a restricted zone. The final list will be agreed with the Requirement Provider.
- A central database for employee records, camera and zone details, sightings, and suspicious events.
- A secure web dashboard with login, a home summary, employee search and profile, alert review, and report generation.
- Security and privacy features: role-based access, audit logs, and data retention limits.
- Testing on selected CCTV footage, project documentation (SRS, design, test, and user documents), and a final demonstration.

Out of Scope and Limitations

- **Installing or replacing cameras:** Only existing cameras will be used. Recognition quality depends on camera resolution, angle, and lighting.
- **People whose face is not visible:** The system will not use gestures, clothes, or body features to identify people.
- **Automatic disciplinary action:** The system only gives alerts for review and will not take any action against employees.
- **Visitors and contractors:** People who are not enrolled will be labeled “Unknown” and will not be identified.
- **Full deployment:** The system will be built and tested on selected cameras or recorded clips only, not deployed at full scale in an organization.
- **Learning-based activity detection:** Suspicious activities will be found with simple rules; training a model on labeled videos is left for future work.

5. STAKEHOLDER IDENTIFICATION:

The people and groups who will use, manage, or be affected by VisionGuard are identified in Table 1.

Table 1: Stakeholders and Their Roles in the Proposed System

Stakeholder	Role	Interaction with System
Security Staff (Operators)	Primary end users	Log in to the dashboard, search employees by Employee ID or name, view sightings, review alerts with snapshot evidence, and mark each event as Pending, Confirmed, or False Alarm.
Security Manager	Supervisor and decision maker	Monitors the summary of events, oversees alert review, creates and exports reports, and agrees the suspicious activity rules, zones, and working hours with the project team.
System Administrator	System manager	Manages user accounts and roles, enrolls employees, configures cameras, zones, and thresholds, and provides access to camera streams and servers.
Requirement Provider (Mr. Ammar Ahmad Khan)	Project supervisor	Provides project information, domain knowledge, and approvals for data and camera access; validates requirements during sprint reviews and accepts the final system.
Enrolled Employees	Data subjects	Do not use the system directly. Their approved face data and Employee ID are enrolled, and they are recognized when they pass in front of the selected cameras. Their privacy is protected by the system.
Visitors and Contractors	Non-enrolled people	No direct interaction. If seen by a camera, they are labeled “Unknown” and are not identified.
Project Team	Developers	Aqsa Aziz, Iqra Gul, Aliya Ashraf, and Areeba Tahir analyze requirements, design, develop, test, and document the system.
Ms. Asiya Batool	Evaluator	Reviews and approves the proposal and project deliverables and gives feedback.

6. SOFTWARE DEVELOPMENT METHODOLOGY:

6.1 Selected Methodology and Justification

The project will follow an Agile approach based on Scrum [6], [7], using an iterative and incremental model. Scrum is the Agile framework that organizes the work, while iterative and incremental describes how the product is built. The work is iterative because the same short cycle (plan, design, implement, test, review) is repeated in every two-week sprint, and each cycle refines the requirements and the system using feedback. It is incremental because every sprint adds a new working part of the system (for example video processing, face recognition, rules, or the dashboard) that is tested and integrated with the earlier parts, so a working system is available early. This is different from a waterfall model, where all requirements are fixed at the start and the phases of analysis, design, implementation, and testing follow one another only once.

This approach is suitable for VisionGuard for five reasons. First, the requirements are not complete at the start: the cameras, zones, activities, and thresholds will be refined after discussion with the Requirement Provider, and Agile accepts such change when it is small, feasible, and does not affect the core of the project. Second, the face recognition and rule components need experiments and tuning on real footage, which fits short iterations with frequent testing. Third, the Requirement Provider can give feedback regularly on working increments instead of waiting until the end. Fourth, the modular structure of the system (video, recognition, tracking, rules, database, and dashboard) allows team members to work on separate increments in parallel. Fifth, the project must be finished within one semester, and time-boxed sprints with a prioritized backlog ensure that the most important features are built first and that the deadline is met even if low-priority items have to be dropped.

6.2 Development Phases

The project is organized as a short initial sprint (Sprint 0) followed by six two-week development sprints (Sprints 1–6). The major phases are shown in Table 2. They are not completed once and closed, as in a waterfall model: a baseline of the requirements and the architecture is prepared in Sprint 0 and is then refined in every sprint, while analysis, design, implementation, and testing are repeated for the features of each sprint.

Table 2: Development Phases, Activities, and Deliverables

Phase	Main Activities	Key Deliverables	Timeline
Planning	Define the project plan, team roles, feasibility, risks, and methodology; create the initial product backlog. Planning is repeated at the start of every sprint (sprint planning).	Project proposal, project plan, initial backlog, sprint plan	Sprint 0 (8–25 Oct 2026), then every sprint
Analysis	Literature review; meetings with the Requirement Provider; review of the camera environment; agree on activities, zones, and working hours; write the baseline requirements. Requirements are reviewed and refined in every sprint, and small feasible changes are added to the backlog.	Baseline SRS, use cases, approved camera and zone list, updated backlog	Baseline in Sprint 0; refined in Sprints 1–6
Design	Outline the system architecture, data model, and API in Sprint 0; then design the user interface, rules, and components of each increment just before building it.	Architecture outline, ER diagram, UI mockups, design notes per sprint	Outline in Sprint 0; detailed in Sprints 1–5

Phase	Main Activities	Key Deliverables	Timeline
Implementation	Build the system in six increments: (1) video ingestion, person detection, and database; (2) face recognition and employee enrolment; (3) tracking and rule engine; (4) alerts and web dashboard; (5) reports, security and privacy, and integration; (6) fixes and final delivery.	Working, tested increment after every sprint	Sprints 1–6 (26 Oct 2026 – 15 Jan 2027)
Testing	Unit testing in every sprint; integration testing at the end of each increment; evaluation of recognition accuracy, rule precision and recall, speed, and alert delay on recorded clips and staged events.	Test plan, test report, evaluation report	Every sprint; system evaluation in Sprints 5–6
Delivery	User acceptance testing with the Requirement Provider, fixes, user manual, final report, and demonstration of the prototype.	Prototype, user manual, final report	Sprint 6 (4–15 Jan 2027)

6.3 Team Organization, Monitoring, and Feedback

The team will organize and monitor its work as follows:

- **Roles:** The Requirement Provider (the project supervisor) acts as the Product Owner who sets priorities and accepts the results. One team member acts as Scrum Master, and the four team members form the development team. The course instructor, Ms. Asiya Batool, guides and evaluates the project.
- **Sprints:** Work is done in two-week sprints (Sprint 0 is a short start-up sprint from 8 to 25 October 2026). Each sprint has sprint planning at the start, short stand-up meetings during the sprint, and a sprint review and retrospective at the end.
- **Planning and tracking:** Requirements are written as user stories in a product backlog. Tasks and progress are tracked on a GitHub Projects board, and a burndown chart shows the progress of each sprint.
- **Feedback:** A working increment is demonstrated to the Requirement Provider in the sprint review at the end of every sprint, where the requirements are validated. The feedback is added to the backlog as new or changed items and is prioritized in the next sprint planning.
- **Quality control:** Every increment is tested before its sprint review, and code is reviewed through pull requests before it is merged.

6.4 Change Management

Requirements are not frozen after the first sprint, but changes are controlled. Every requested change is recorded in the product backlog, checked by the team for effort and effect on the core of the project, and discussed with the Requirement Provider in sprint planning. Changes are handled in two ways:

- **Accepted changes:** Small and feasible changes that do not affect the core of the project, such as thresholds, zones, working hours, alert wording, or dashboard layout, are accepted and implemented in the current or next sprint. The rules and thresholds are configurable, so these changes can be handled quickly.
- **Changes that need discussion:** Significant changes, such as new modules, a different identification method, or anything that puts the January 2027 deadline at risk, will be discussed with the Requirement Provider and the project team before implementation, as stated in the Requirement Provider Agreement. If such a change is approved, a lower-priority item is moved out of the sprint scope or to future work.

All source code and documents are managed with Git, and the revision history of this document is updated whenever its content changes.

6.5 Tentative Schedule and Gantt Chart

The development schedule is planned for one semester, from 8 October 2026 to 15 January 2027. Figure 1 shows the major activities, milestones, and expected timeline as a sprint-based plan. The five course milestones are shown on the chart: the project proposal in Week 4 (9 October 2026), the Software Requirements Specification (SRS) in Week 8 (6 November 2026), the Software Design Specification (SDS) in Week 12 (4 December 2026), the working prototype in Week 15 (25 December 2026), and the project presentation in Week 16 (1 January 2027). Sprint 0 (8–25 October 2026) prepares the project plan, the baseline SRS, and the initial product backlog. Sprints 1–6 are two-week sprints, and in each of them the team repeats a short cycle of sprint planning and requirement refinement, design and implementation, and testing, review, and retrospective, so the requirements are revisited throughout the project rather than only at the start. A sprint review with the Requirement Provider is held at the end of every sprint, and documentation continues throughout the project. The schedule is tentative and may change with the Requirement Provider’s feedback and the availability of camera access and permissions.

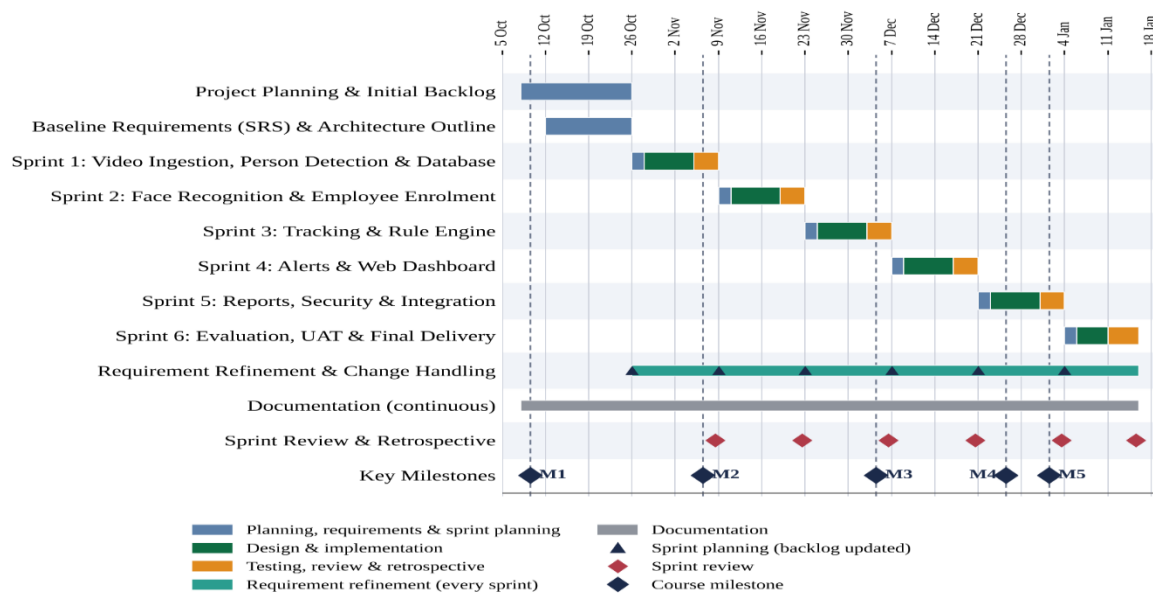


Figure 1: Tentative Project Gantt Chart (8 October 2026 – 15 January 2027)

7. TOOLS AND TECHNOLOGIES:

The tools below are the initial plan. The final choice may change during development, depending on the requirements, the CCTV footage, and the test results.

Table 3: Proposed Tools and Technologies

#	Tools and Technology	Purpose
#	Tools and Technology	Purpose
1	Python	Main language for the AI pipeline, rule engine, and backend logic.
2	JavaScript and React	Development of the web dashboard.
3	FastAPI	Backend REST API and real-time alert delivery to the dashboard.
4	PostgreSQL	Central database for employees, cameras, zones, sightings, and suspicious events.
5	OpenCV and FFmpeg	Reading CCTV streams (RTSP) and extracting frames.
6	Ultralytics YOLO [8]	Detecting people in each processed frame.
7	ByteTrack [9]	Tracking each person across frames to keep identity, position, and time spent in an area.
8	InsightFace (SCRFD [10] and ArcFace [11])	Face detection and alignment, and face embeddings for recognition.
9	Face quality check (custom) [4]	Skipping small, blurry, or side-facing faces before recognition.
10	FAISS [12]	Fast similarity search between face embeddings and enrolled employees.
11	PyTorch and ONNX Runtime	Running the AI models, with ONNX Runtime if faster execution is needed.
12	scikit-learn and Matplotlib	Calculating evaluation measures (accuracy, precision, recall, ROC) and plotting results.
13	CVAT	Labeling video clips for testing, if needed.
14	pytest	Automated unit and integration tests for the backend and rule engine.
15	Figma	User interface design and prototypes of the dashboard.
16	draw.io	UML, architecture, and ER diagrams for the requirements and design documents.
17	Docker	Consistent set-up of the development and deployment environment.
18	Git, GitHub, and GitHub Projects	Version control, code review, product backlog, and sprint tracking.
19	LFW dataset [13]	Early checks of the face model; the main tests use approved footage recorded from the selected cameras and from consenting volunteers.
20	Existing CCTV or IP cameras	Source of the video streams (or recorded clips); no new hardware is required.

REFERENCES

- [1] H. L. Gururaj, B. C. Soundarya, S. Priya, J. Shreyas, and F. Flammini, “A comprehensive review of face recognition techniques, trends, and challenges,” *IEEE Access*, vol. 12, pp. 107903–107926, 2024, doi: 10.1109/ACCESS.2024.3424933.
- [2] W. Sultani, C. Chen, and M. Shah, “Real-world anomaly detection in surveillance videos,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 6479–6488.
- [3] A. Fola-Rose, E. Solomon, K. Bryant, and A. Woubie, “A systematic review of facial recognition methods: Advancements, applications, and ethical dilemmas,” in *Proc. IEEE Int. Conf. Inf. Reuse Integr. Data Sci. (IRI)*, 2024, doi: 10.1109/IRI62200.2024.00070.
- [4] A. A. Ibrahim et al., “A lightweight face quality assessment framework to improve face verification performance in real-time screening applications,” arXiv:2507.15961, 2025.
- [5] J. Núñez, Z. Li, S. Escalera, and K. Nasrollahi, “Identifying loitering behavior with trajectory analysis,” in *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis. Workshops (WACVW)*, 2024, pp. 251–259, doi: 10.1109/WACVW60836.2024.00033.
- [6] K. Beck et al., “Manifesto for Agile Software Development,” Agile Alliance, 2001. [Online]. Available: <https://agilemanifesto.org>
- [7] K. Schwaber and J. Sutherland, “The Scrum Guide: The definitive guide to Scrum: The rules of the game,” Scrum.org, Nov. 2020. [Online]. Available: <https://scrumguides.org>
- [8] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 779–788.
- [9] Y. Zhang et al., “ByteTrack: Multi-object tracking by associating every detection box,” in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2022, pp. 1–21.
- [10] J. Guo, J. Deng, A. Lattas, and S. Zafeiriou, “Sample and computation redistribution for efficient face detection,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2022.
- [11] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, “ArcFace: Additive angular margin loss for deep face recognition,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 4690–4699.
- [12] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021.
- [13] G. B. Huang, M. Ramesh, T. Berg, and E. Learned-Miller, “Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments,” Univ. Massachusetts, Amherst, MA, USA, Tech. Rep. 07-49, Oct. 2007.